MNIST DIGIT RECOGNITION

BY: MUHAMMAD ANAS RATHORE

FULL NAME: MUHAMMAD ANAS RATHORE
DOMAIN: MACHINE LEARNING
EMAIL ADDRESS: anaskhanot7@gmail.com

ACKNOWLEDGEMENT

*First and foremost, all praise and gratitude are due to **Allah SWT**, whose guidance, blessings, and mercy enabled me to complete this project successfully. Without **His** support, none of this work would have been possible.*

*I would like to express my deepest gratitude to my **parents**, whose unwavering love, encouragement, and sacrifices have been a constant source of strength and motivation throughout my academic journey. Their belief in my abilities has inspired me to strive for excellence in every endeavor.*

*I would also like to sincerely acknowledge **Miss Mahrose Fatima**, **Asma Yahya**, and **Asma Qaiser**, my mentors, for being inspiring guides during my studies at Iqra University. Their dedication to teaching and academic guidance has positively influenced my learning journey.*

Finally, I am grateful to all my friends, colleagues, and everyone who directly or indirectly contributed to the successful completion of this project. Their encouragement and support made this journey smoother and more enjoyable.

This work stands as a result of the guidance, support, and inspiration I received from all the above-mentioned individuals, and I am sincerely thankful to each of them.

TABLE OF CONTENT

Problem Statement

The task was designed to provide practical experience in the fields of Artificial Intelligence, Machine Learning, Deep Learning, and Computer Vision by developing an intelligent system capable of recognizing handwritten digits.

Handwritten digit recognition is one of the fundamental and widely studied problems in the field of Computer Vision and Artificial Intelligence. While humans can easily identify handwritten numbers regardless of differences in writing styles, computers require advanced algorithms and learning techniques to perform the same task accurately. Variations in handwriting, stroke thickness, image quality, digit orientation, and writing patterns make this problem challenging for traditional computer-based systems.

To address this challenge, this project focuses on developing a deep learning based handwritten digit recognition system using the MNIST dataset. The system is designed to analyze grayscale images of handwritten digits and classify them into one of the ten digit classes ranging from **0 to 9**.

The primary goal of this project is to build an efficient, reliable, and accurate handwritten digit recognition system that demonstrates the practical application of deep learning techniques in solving image classification problems.

Objective

The main objectives of this project are:

1. To analyze and preprocess the MNIST handwritten digit dataset.
2. To build and compare Artificial Neural Network (ANN) and Convolutional Neural Network (CNN) models.
3. To evaluate model performance using different evaluation metrics.
4. To visualize dataset characteristics through graphs and statistical analysis.
5. To identify misclassified images and understand model behavior.
6. To deploy the best-performing model using Streamlit.
7. To create a user-friendly web application for handwritten digit prediction.

Description

This project focuses on developing a Handwritten Digit Recognition System using Deep Learning techniques.

The MNIST dataset was used as the primary data source. The dataset contains grayscale images of handwritten digits ranging from 0 to 9. Each image consists of 28 × 28 pixels and represents a single handwritten digit.

The project begins with data loading, cleaning, and exploratory data analysis. Various visualizations were generated to understand the dataset distribution and image characteristics.

After preprocessing the data, two deep learning models were developed:

1. Artificial Neural Network (ANN)

The ANN model consists of:

1. Input Flatten Layer
2. Dense Layer (256 neurons)
3. Dropout Layer
4. Dense Layer (128 neurons)
5. Dropout Layer
6. Dense Layer (64 neurons)
7. Output Layer (10 classes)

2. Convolutional Neural Network (CNN)

The CNN model consists of:

1. Convolution Layer
2. Max Pooling Layer
3. Convolution Layer
4. Max Pooling Layer
5. Flatten Layer
6. Dense Layer
7. Dropout Layer
8. Softmax Output Layer

The performance of both models was compared using various evaluation metrics.

Finally, the CNN model was deployed through a Streamlit application that allows users to upload handwritten digit images and receive predictions with confidence scores.

Methodology

The development of the Handwritten Digit Recognition System followed a systematic process consisting of data preparation, analysis, model development, evaluation, and application implementation. Each phase was carefully executed to ensure accurate digit classification and reliable model performance.

1. Data Collection

The first step involved obtaining the MNIST dataset, which contains thousands of handwritten digit images ranging from 0 to 9. The dataset was loaded from CSV files and prepared for further analysis and processing.

1. Loaded training and testing datasets using Pandas.
2. Extracted image pixel values and corresponding labels.
3. Converted the data into a suitable format for analysis.
4. Verified dataset dimensions and structure.
5. Examined the number of samples available for each dataset.

2. Data Cleaning

Data cleaning was performed to ensure the dataset was free from inconsistencies and ready for model training. Although MNIST is a well-structured dataset, validation checks were conducted to maintain data quality.

1. Converted all values into numeric format.
2. Removed rows containing invalid or missing values.
3. Checked for duplicate images.
4. Verified the absence of corrupted records.
5. Identified all-black and all-white images.
6. Confirmed dataset integrity before preprocessing.

3. Exploratory Data Analysis

Exploratory Data Analysis was conducted to understand the characteristics of the dataset and visualize important patterns. Various graphs and visualizations were generated to gain insights into the image data.

1. Displayed random handwritten digit samples.
2. Analyzed class distribution of digits.
3. Generated pixel intensity histograms.
4. Created box plots for pixel value analysis.
5. Produced KDE (Kernel Density Estimation) plots.
6. Visualized pixel variance using heatmaps.
7. Displayed one sample image from each digit class.
8. Examined overall dataset statistics and summaries.

4. Data Preprocessing

Data preprocessing transformed the raw image data into a format suitable for deep learning models. This step improved model efficiency and training performance.

1. Normalized pixel values from 0–255 to 0–1.
2. Reshaped images into CNN-compatible format (28×28×1).
3. Applied one-hot encoding to class labels.
4. Split data into training and validation sets.
5. Prepared datasets for ANN and CNN training.

5. Data Augmentation

Data augmentation was applied to increase the diversity of training samples and improve model generalization. This helps the model recognize handwritten digits in different writing styles and orientations.

1. Applied image rotation transformations.
2. Performed zoom operations.
3. Applied width shifting.
4. Applied height shifting.
5. Used shear transformations.
6. Generated augmented image samples during training.

6. Model Development

Two deep learning models (Artificial Neural Network and Convolutional Neural Network) were developed and trained to compare their performance on the handwritten digit recognition task. Both models were designed using TensorFlow and Keras.

1. Developed an Artificial Neural Network (ANN).
2. Implemented multiple dense layers in ANN.
3. Added dropout layers to reduce overfitting.
4. Developed a Convolutional Neural Network (CNN).
5. Added convolution and pooling layers.
6. Compiled both models using the Adam optimizer.
7. Trained models using training and validation datasets.

7. Model Evaluation

The trained models were evaluated using various performance metrics to measure prediction accuracy and classification quality. The results of both models were compared to identify the better-performing solution.

1. Created confusion matrices.
2. Generated classification reports.
3. Compared ANN and CNN performance.

4. Analyzed training and validation accuracy.
5. Evaluated training and validation loss.
6. Visualized model comparison graphs.
7. Identified misclassified digit images.

8. Advanced Visualization and Analysis

Additional visualization techniques were used to better understand feature representation and model behavior. These visualizations provided deeper insights into the learned patterns within the dataset.

1. Generated PCA projections.
2. Created t-SNE visualizations.
3. Visualized CNN feature maps.
4. Analyzed extracted image features.
5. Studied digit clustering patterns.

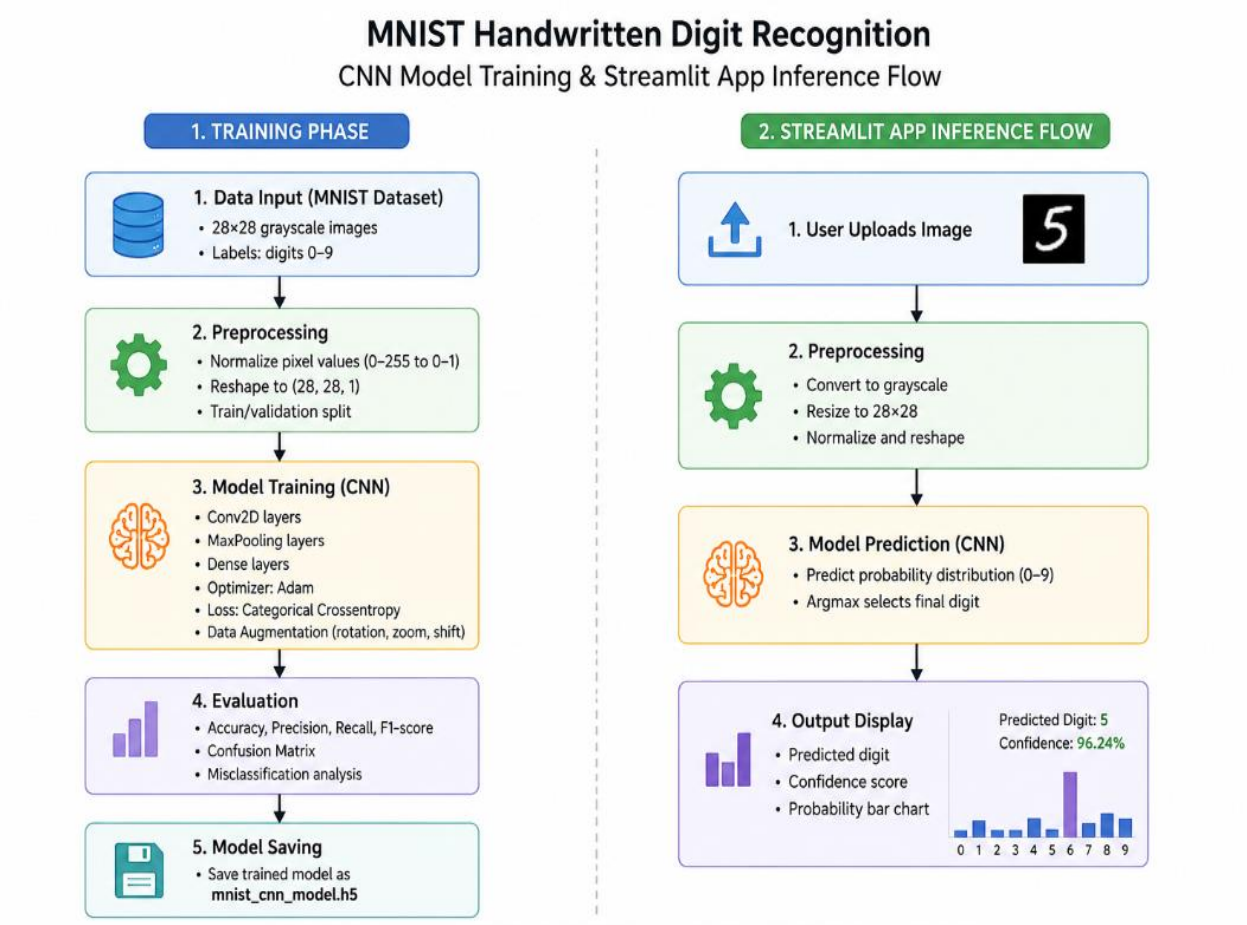
9. Streamlit Application Implementation

After selecting the CNN model as the best-performing model, it was integrated into a Streamlit application. The application provides an interactive interface for testing handwritten digit predictions.

1. Saved the trained CNN model.
2. Developed a Streamlit-based user interface.
3. Enabled image upload functionality.
4. Implemented image preprocessing.
5. Generated real-time digit predictions.
6. Displayed prediction confidence scores.
7. Visualized probability distributions for all digit classes.
8. Executed the application locally using Streamlit.

Note: The application was run locally using Streamlit for demonstration and testing purposes and was not deployed on any online hosting platform.

Flow Chart



Note: This image has been generated by AI.

Python Implementation

The Handwritten Digit Recognition System was entirely developed in Python. Various libraries were utilized throughout the project to perform data handling, visualization, deep learning model development, evaluation, image processing, and application development. Each library contributed to a specific stage of the project workflow, enabling efficient implementation of the complete recognition system.

Libraries and Their Contributions

NumPy

1. Used for numerical computations and array operations.
2. Assisted in image reshaping, normalization, and matrix manipulation.

Pandas

1. Used for loading and managing the MNIST dataset.
2. Helped perform data cleaning, exploration, and preprocessing tasks.

Matplotlib

1. Utilized to create visual representations of the dataset and model performance.
2. Generated histograms, training curves, sample image plots, and comparison graphs.

Seaborn

1. Used to create advanced statistical visualizations.
2. Assisted in generating heatmaps, confusion matrices, and distribution plots.

TensorFlow / Keras

1. Served as the primary deep learning framework.
2. Used to build, train, validate, and evaluate ANN and CNN models.

Scikit-Learn

1. Provided evaluation metrics such as Accuracy, Precision, Recall, and F1-Score.
2. Used for dimensionality reduction techniques including PCA and t-SNE.

PIL (Python Imaging Library)

1. Used to preprocess uploaded images before prediction.
2. Assisted with image resizing, grayscale conversion, and inversion operations.

Streamlit

1. Used to create an interactive graphical user interface.
2. Enabled users to upload handwritten digit images and obtain predictions in real time.

Major Functionalities Implemented

The Python implementation includes the following major functionalities:

1. Dataset Loading and Validation
2. Data Cleaning and Verification
3. Exploratory Data Analysis (EDA)
4. Image Normalization
5. Data Augmentation
6. ANN Model Development
7. CNN Model Development
8. Model Training and Validation
9. Performance Evaluation
10. Confusion Matrix Generation
11. Classification Report Generation
12. PCA and t-SNE Visualization
13. CNN Feature Map Visualization
14. Model Saving and Loading
15. Streamlit-Based User Interface
16. Real-Time Digit Prediction

Output, Charts and Graphs

The system generates the following outputs:

```
📊 MNIST DATASET OVERVIEW
============================================================
Training samples : 60,000
Testing samples  : 10,000
Image dimensions : 28 x 28 (grayscale)
Number of classes: 10 (digits 0-9)
Pixel value range: [0, 255]
Data type        : int64
============================================================
🔷 TRAIN HEAD
   label  pixel1  pixel2  pixel3  pixel4  pixel5  pixel6  pixel7  pixel8  \
1    5.0     0.0     0.0     0.0     0.0     0.0     0.0     0.0     0.0
2    0.0     0.0     0.0     0.0     0.0     0.0     0.0     0.0     0.0
3    4.0     0.0     0.0     0.0     0.0     0.0     0.0     0.0     0.0
4    1.0     0.0     0.0     0.0     0.0     0.0     0.0     0.0     0.0
5    9.0     0.0     0.0     0.0     0.0     0.0     0.0     0.0     0.0

   pixel9  ...  pixel775  pixel776  pixel777  pixel778  pixel779  pixel780  \
1     0.0  ...       0.0       0.0       0.0       0.0       0.0       0.0
2     0.0  ...       0.0       0.0       0.0       0.0       0.0       0.0
3     0.0  ...       0.0       0.0       0.0       0.0       0.0       0.0
4     0.0  ...       0.0       0.0       0.0       0.0       0.0       0.0
5     0.0  ...       0.0       0.0       0.0       0.0       0.0       0.0

   pixel781  pixel782  pixel783  pixel784
1       0.0       0.0       0.0       0.0
2       0.0       0.0       0.0       0.0
3       0.0       0.0       0.0       0.0
4       0.0       0.0       0.0       0.0
5       0.0       0.0       0.0       0.0

[5 rows x 785 columns]
============================================================
🔷 TRAIN TAIL
       label  pixel1  pixel2  pixel3  pixel4  pixel5  pixel6  pixel7  pixel8  \
59996    8.0     0.0     0.0     0.0     0.0     0.0     0.0     0.0     0.0
59997    3.0     0.0     0.0     0.0     0.0     0.0     0.0     0.0     0.0
59998    5.0     0.0     0.0     0.0     0.0     0.0     0.0     0.0     0.0
59999    6.0     0.0     0.0     0.0     0.0     0.0     0.0     0.0     0.0
60000    8.0     0.0     0.0     0.0     0.0     0.0     0.0     0.0     0.0

       pixel9  ...  pixel775  pixel776  pixel777  pixel778  pixel779  \
59996     0.0  ...       0.0       0.0       0.0       0.0       0.0
59997     0.0  ...       0.0       0.0       0.0       0.0       0.0
59998     0.0  ...       0.0       0.0       0.0       0.0       0.0
59999     0.0  ...       0.0       0.0       0.0       0.0       0.0
60000     0.0  ...       0.0       0.0       0.0       0.0       0.0

       pixel780  pixel781  pixel782  pixel783  pixel784
59996       0.0       0.0       0.0       0.0       0.0
59997       0.0       0.0       0.0       0.0       0.0
59998       0.0       0.0       0.0       0.0       0.0
59999       0.0       0.0       0.0       0.0       0.0
60000       0.0       0.0       0.0       0.0       0.0

[5 rows x 785 columns]
```

Chart 1: Data set overview

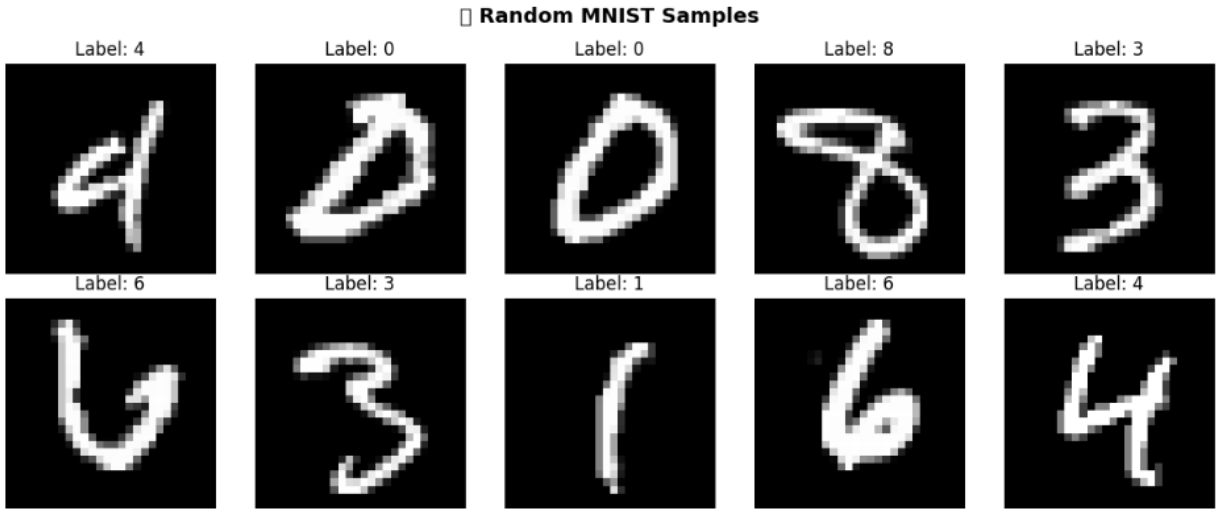


Figure 1: Random MNIST samples from data set

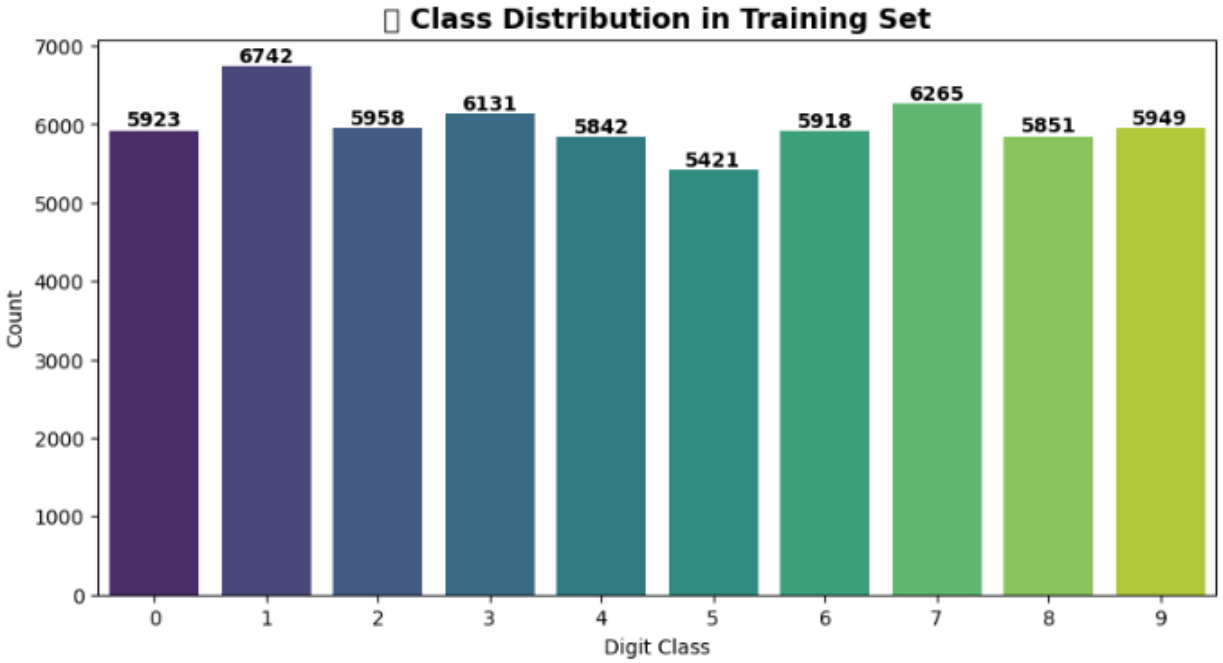


Figure 2: Digits class distribution graph

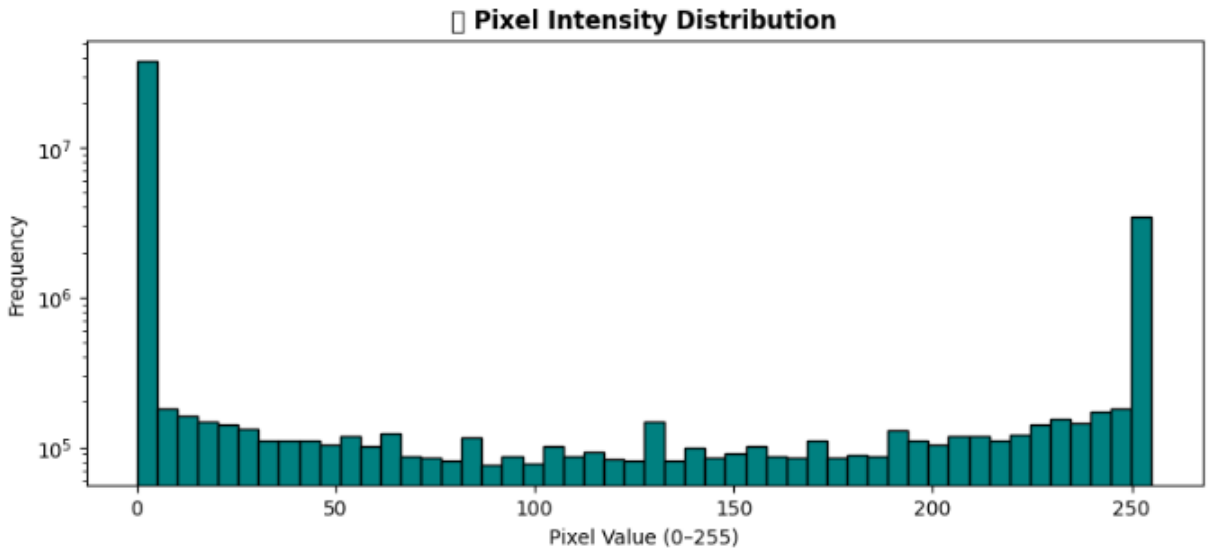


Figure 3: Pixel intensity distribution graph

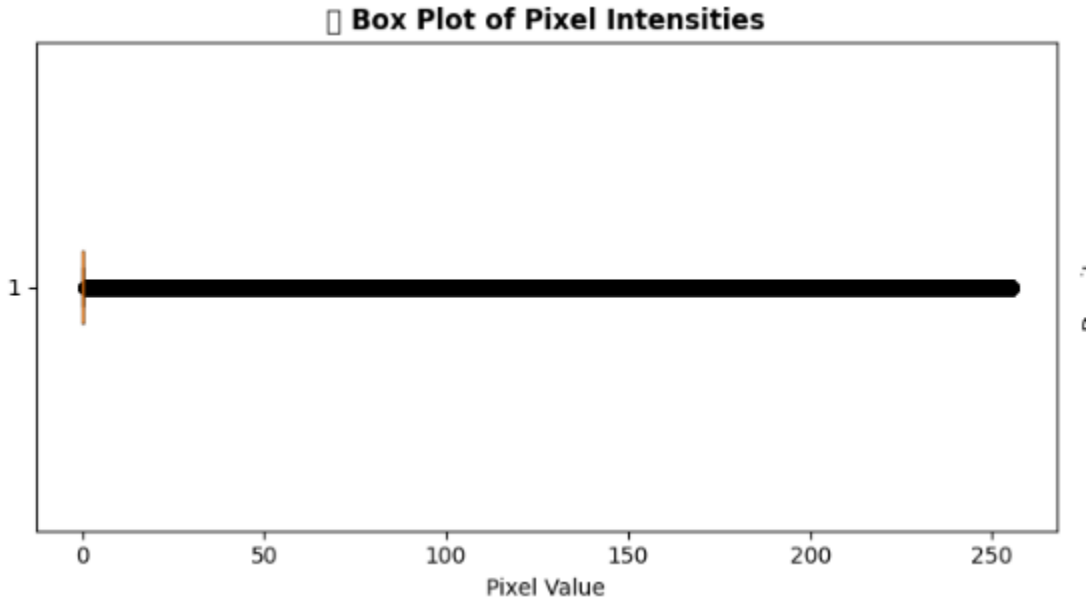


Figure 4: Box plot visualization for pixel intensities.

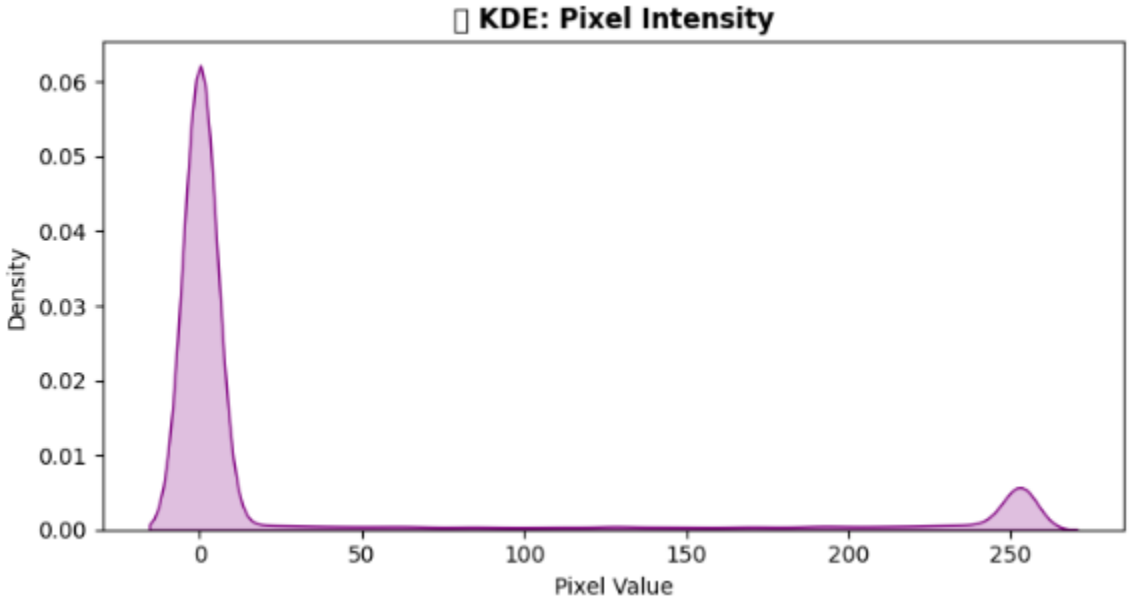


Figure 5: KDE visualization for pixel intensities.

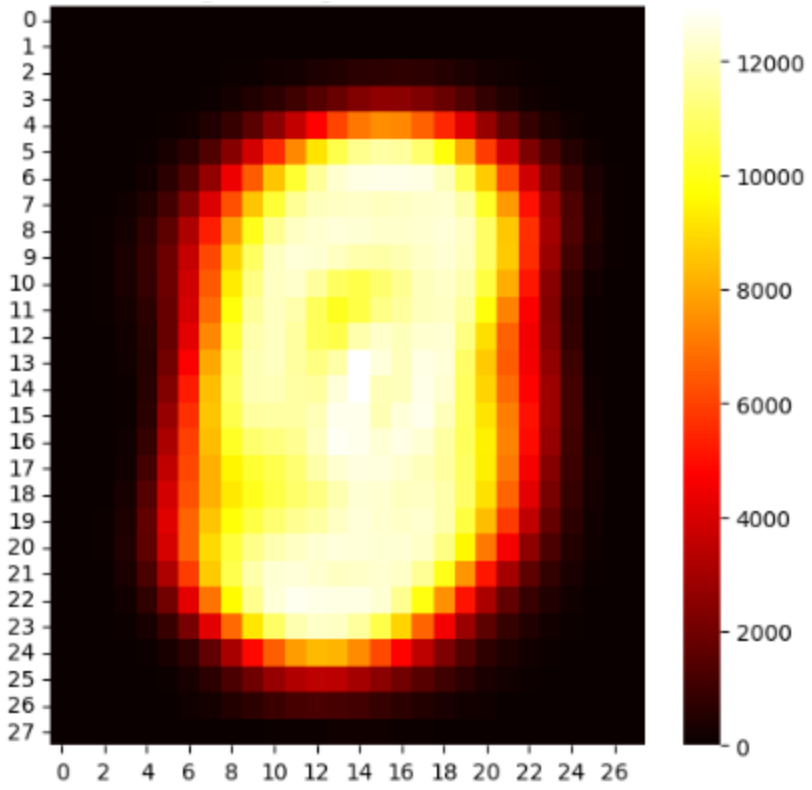


Figure 6: Pixel Variance Heat map

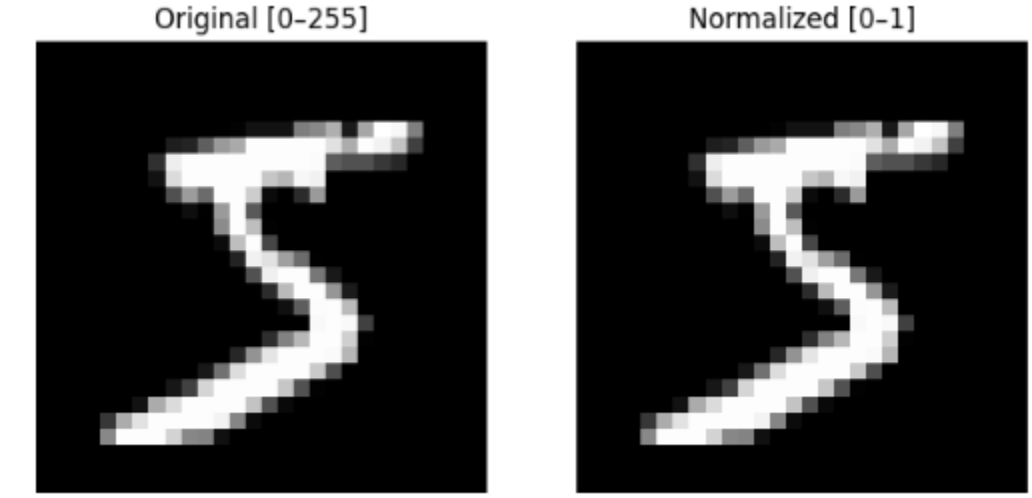


Figure 7: Original vs Normalized image

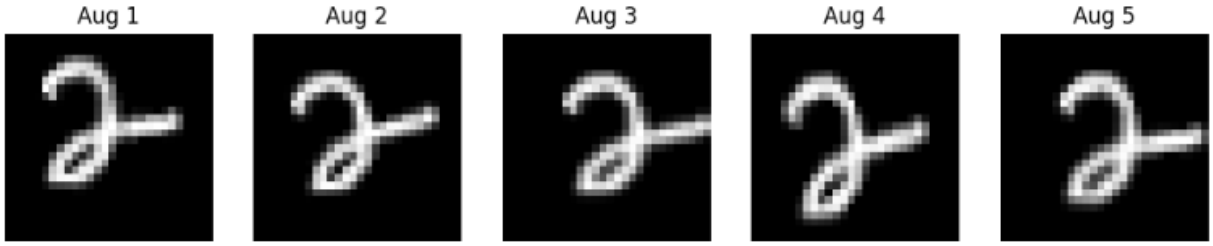


Figure 8: Data Augmentation example

Layer (type)	Output Shape	Param #
flatten (Flatten)	(None, 784)	0
dense (Dense)	(None, 256)	200,960
dropout (Dropout)	(None, 256)	0
dense_1 (Dense)	(None, 128)	32,896
dropout_1 (Dropout)	(None, 128)	0
dense_2 (Dense)	(None, 64)	8,256
dense_3 (Dense)	(None, 10)	650

Total params: 242,762 (948.29 KB)
Trainable params: 242,762 (948.29 KB)
Non-trainable params: 0 (0.00 B)

Chart 2: ANN- Baseline

Layer (type)	Output Shape	Param #
conv2d_2 (Conv2D)	(None, 26, 26, 16)	160
max_pooling2d_2 (MaxPooling2D)	(None, 13, 13, 16)	0
conv2d_3 (Conv2D)	(None, 11, 11, 32)	4,640
max_pooling2d_3 (MaxPooling2D)	(None, 5, 5, 32)	0
flatten_2 (Flatten)	(None, 800)	0
dense_6 (Dense)	(None, 64)	51,264
dropout_3 (Dropout)	(None, 64)	0
dense_7 (Dense)	(None, 10)	650

Total params: 56,714 (221.54 KB)
Trainable params: 56,714 (221.54 KB)
Non-trainable params: 0 (0.00 B)

Chart 3: CNN- Baseline

Model	Accuracy	Precision	Recall	F1 Score	Loss
ANN	0.9802	0.980224	0.9802	0.980194	0.069512
CNN	0.9878	0.987815	0.9878	0.987796	0.038274

Chart 4: Models comparison

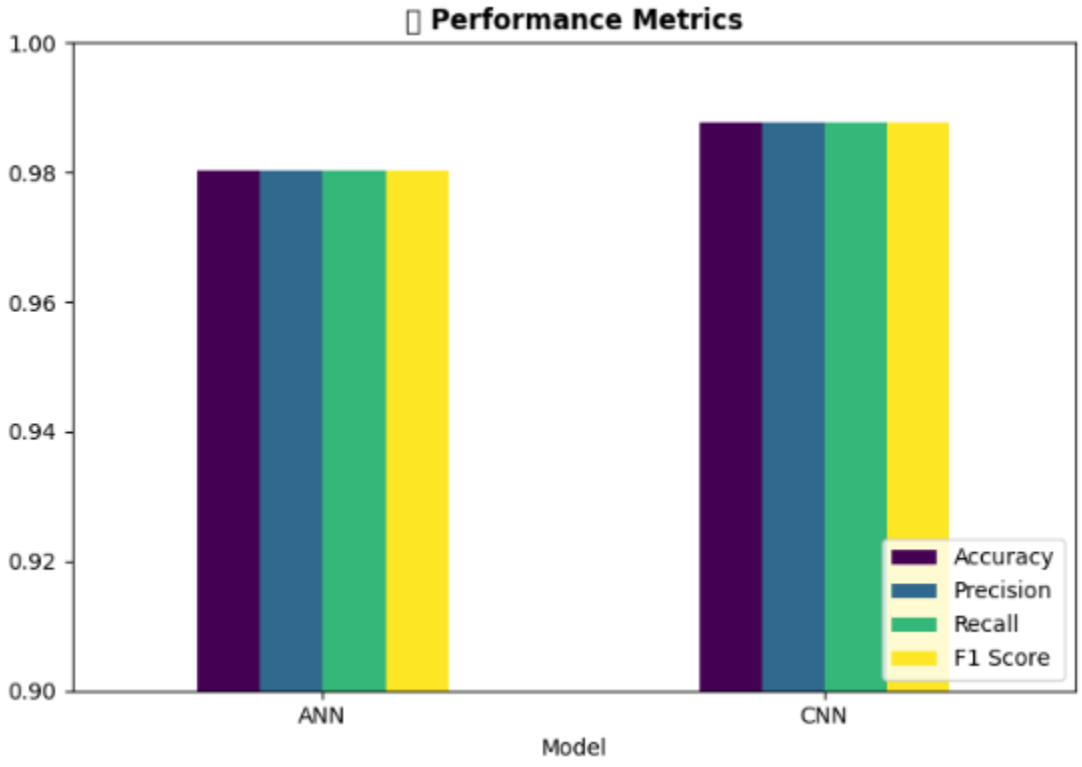


Figure 9: Models performance graph

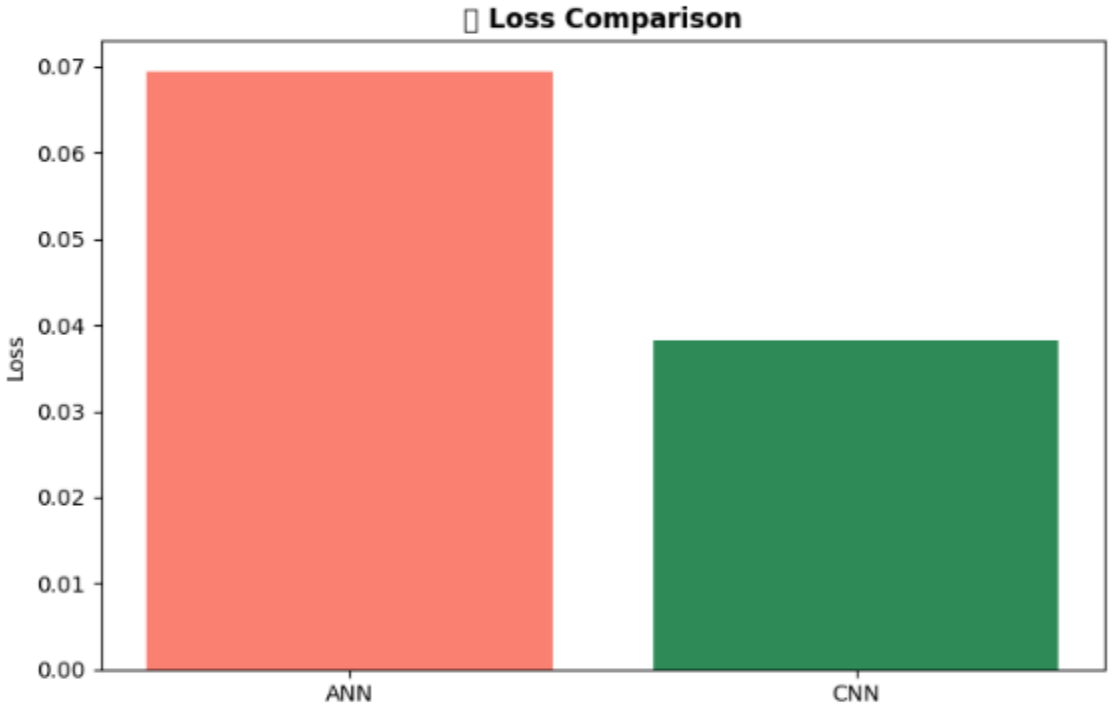


Figure 10: Loss comparison between the models

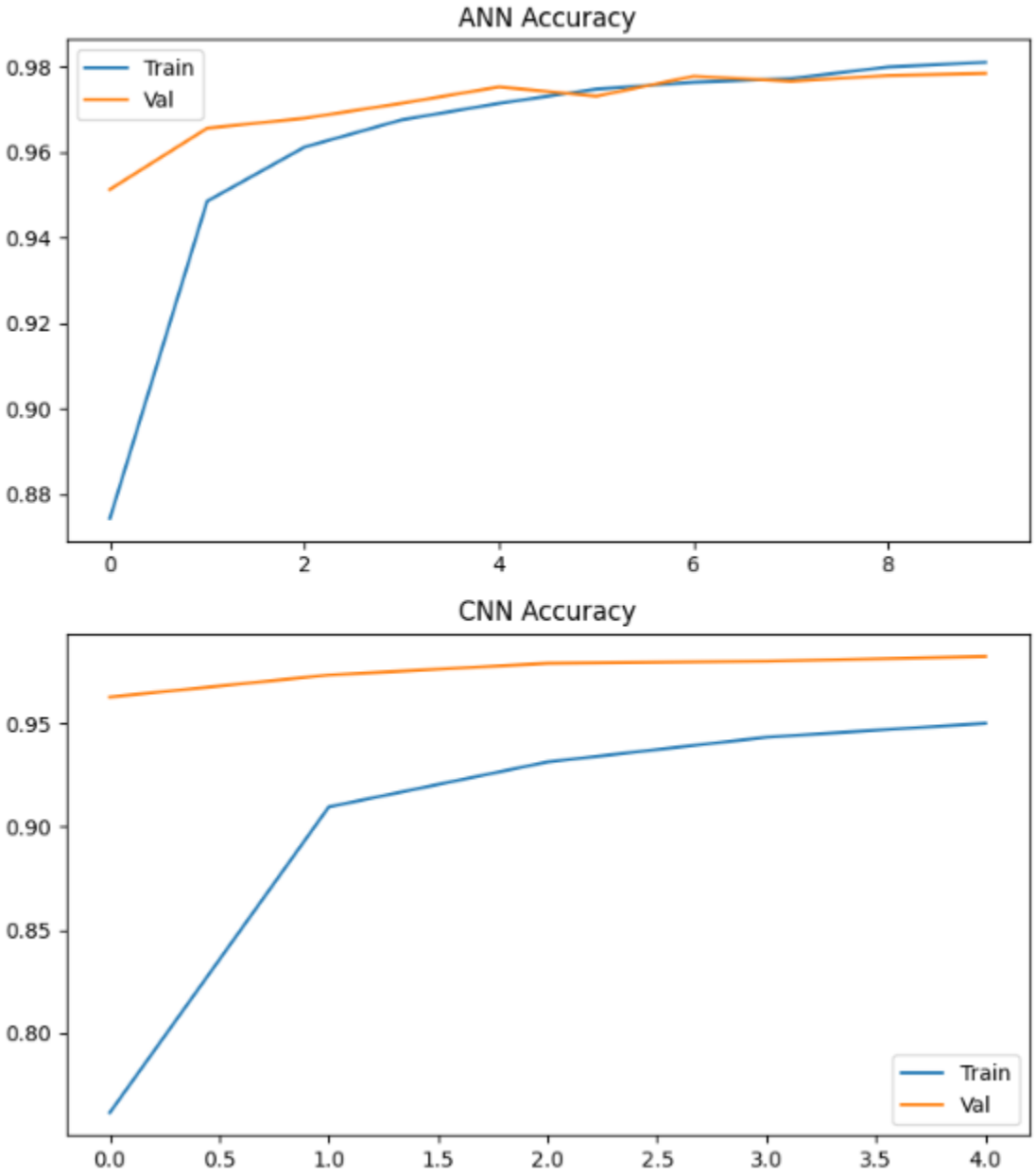


Figure 11: ANN vs CNN accuracy comparison graph

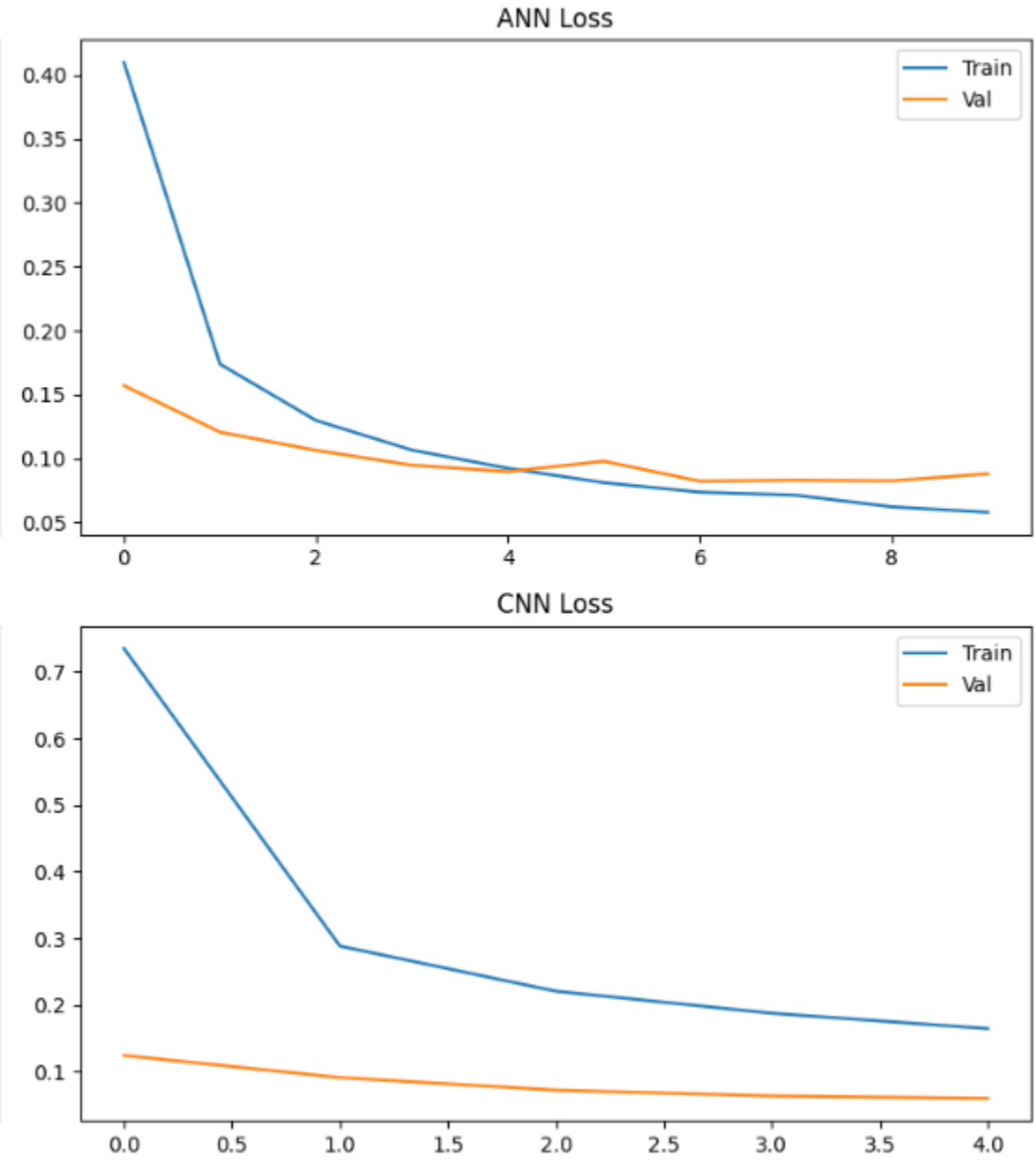


Figure 12: ANN vs CNN loss comparison graph

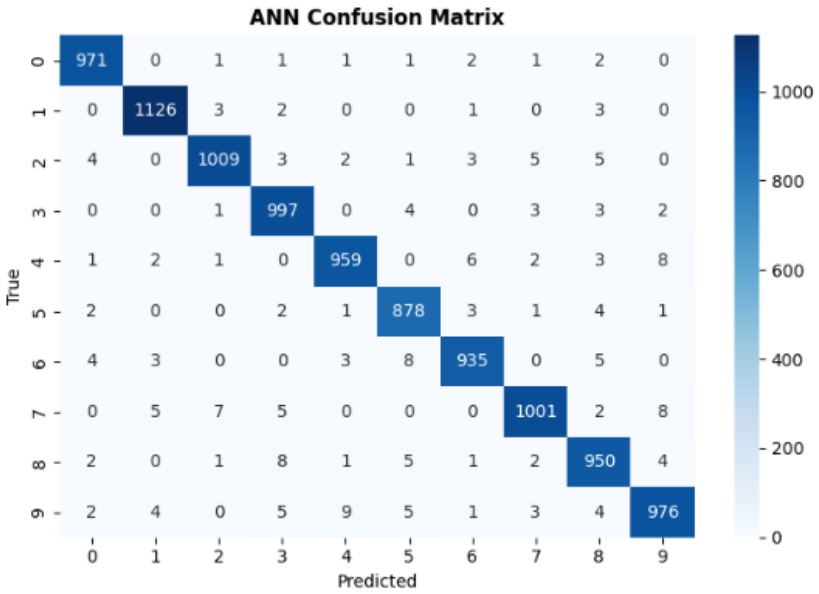


Figure 13: ANN confusion matrix

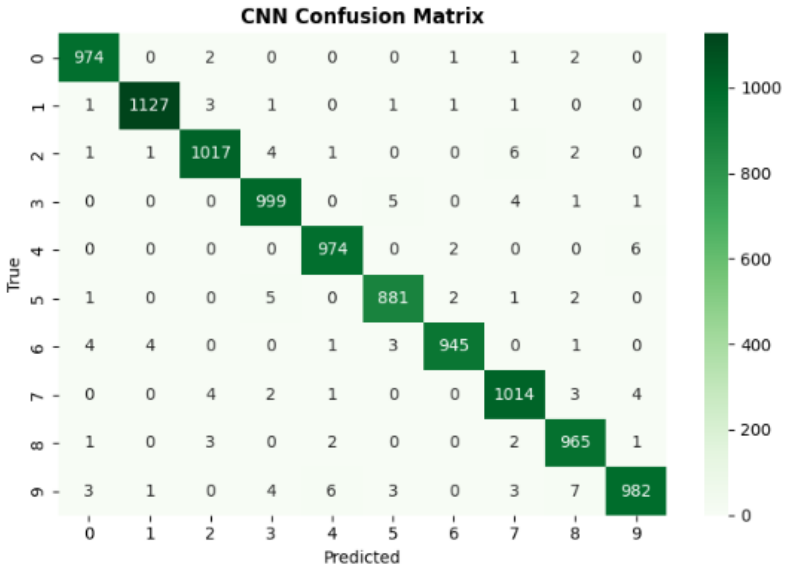


Figure 14: CNN confusion matrix

ANN Classification Report

	precision	recall	f1-score	support
0	0.98	0.99	0.99	980
1	0.99	0.99	0.99	1135
2	0.99	0.98	0.98	1032
3	0.97	0.99	0.98	1010
4	0.98	0.98	0.98	982
5	0.97	0.98	0.98	892
6	0.98	0.98	0.98	958
7	0.98	0.97	0.98	1028
8	0.97	0.98	0.97	974
9	0.98	0.97	0.97	1009
accuracy			0.98	10000
macro avg	0.98	0.98	0.98	10000
weighted avg	0.98	0.98	0.98	10000

CNN Classification Report

	precision	recall	f1-score	support
0	0.99	0.99	0.99	980
1	0.99	0.99	0.99	1135
2	0.99	0.99	0.99	1032
3	0.98	0.99	0.99	1010
4	0.99	0.99	0.99	982
5	0.99	0.99	0.99	892
6	0.99	0.99	0.99	958
7	0.98	0.99	0.98	1028
8	0.98	0.99	0.99	974
9	0.99	0.97	0.98	1009
accuracy			0.99	10000
macro avg	0.99	0.99	0.99	10000
weighted avg	0.99	0.99	0.99	10000

Chart 5: Models performance report

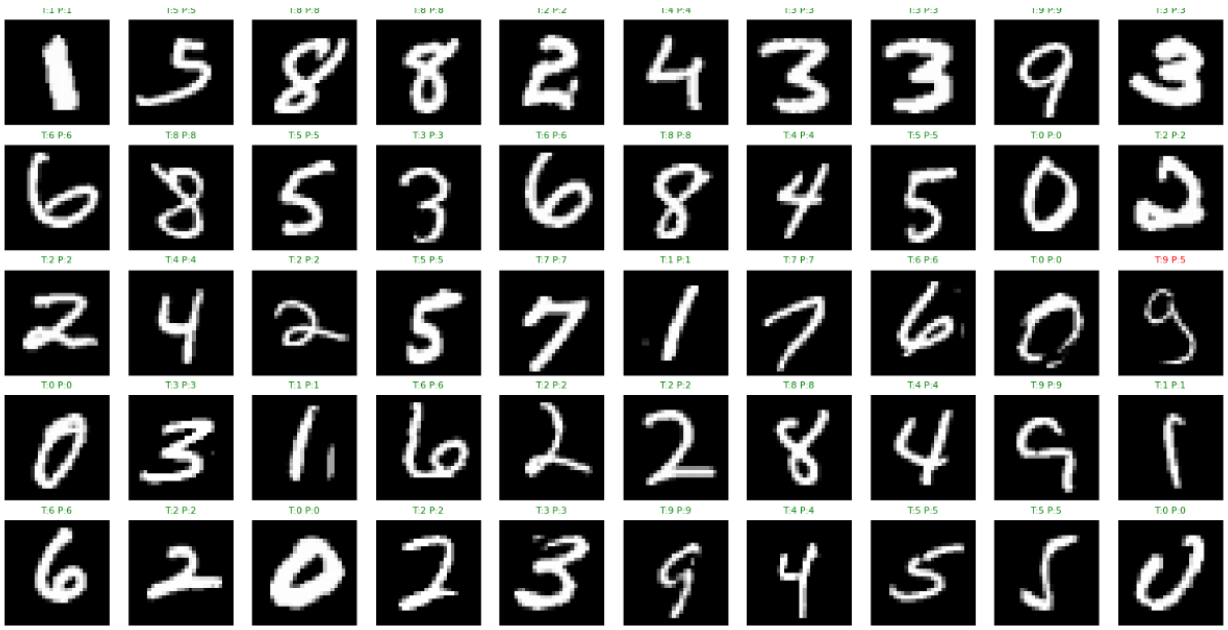


Figure 15: 50 images prediction results

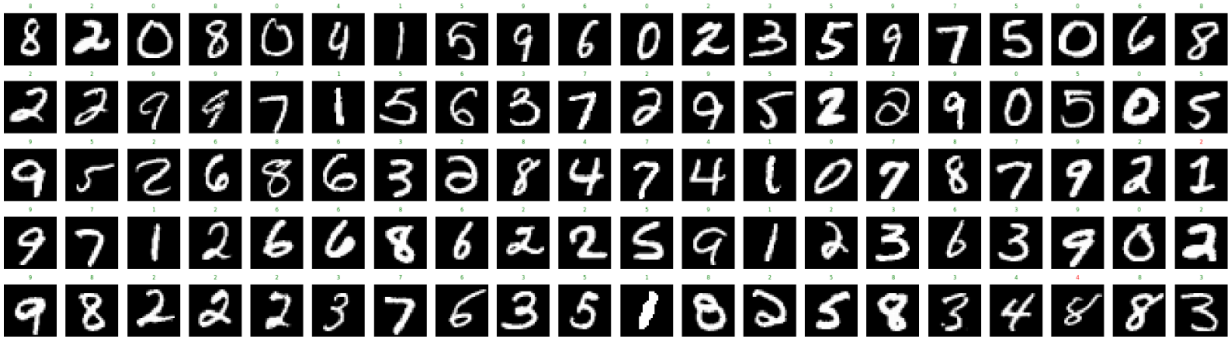


Figure 16: 100 images prediction results

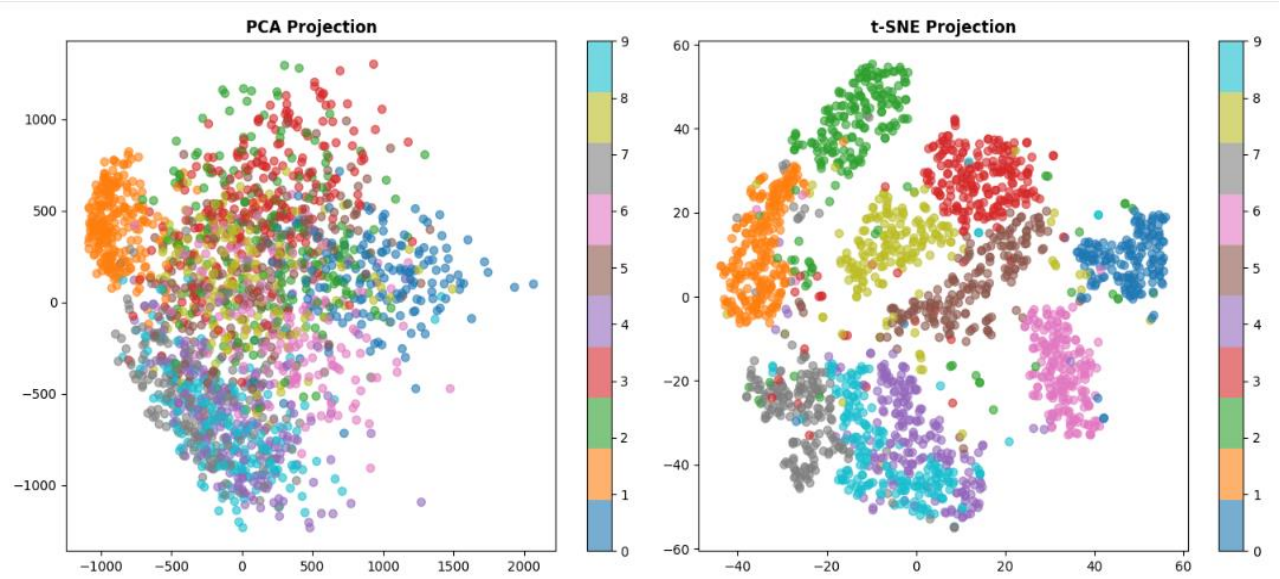


Figure 17: PCA vs t- SNE Projection result

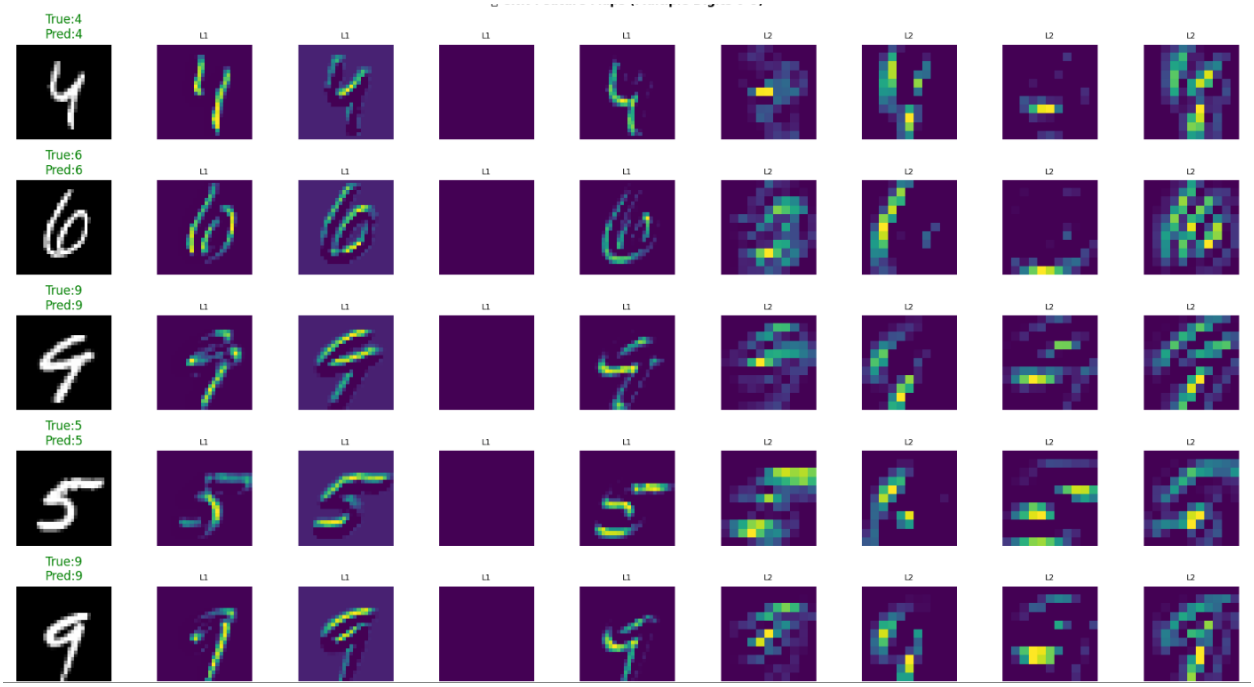


Figure 18: CNN Feature Map results (Random)

Conclusion

This project successfully developed a Handwritten Digit Recognition System using Deep Learning techniques. Both ANN and CNN models were trained and evaluated using the MNIST dataset.

The Convolutional Neural Network demonstrated superior performance due to its ability to automatically learn spatial features from images. The model achieved very high prediction accuracy and effectively classified handwritten digits.

Extensive exploratory data analysis and visualization techniques helped in understanding the dataset characteristics and model behavior. Furthermore, the deployment of the CNN model through Streamlit transformed the trained model into an interactive and user-friendly application.

Overall, this project provided valuable hands-on experience in Artificial Intelligence, Computer Vision, Deep Learning, Data Analysis, and Web Application Development.

Writer's Introduction

Assalam-u-Alaikum,

I am **Muhammad Anas Rathore**, a dedicated student currently pursuing my undergraduate studies in Computer Science. I maintain a strong academic record with a CGPA of **3.9** and continuously strive to expand my technical knowledge while applying innovative technologies to solve real-world challenges.

My primary areas of interest include **Artificial Intelligence, Machine Learning, Deep Learning, Data Science, and Software Development**. Throughout my academic journey, I have worked on multiple practical projects that have strengthened my technical expertise and enhanced my problem-solving abilities. These experiences have enabled me to bridge the gap between theoretical concepts and real-world applications.

In addition to my academic projects, I have gained valuable hands-on experience through internships and collaborative development work, where I contributed to software and AI-based solutions. These experiences have helped me strengthen my technical, analytical, and communication skills while working in professional environments.

As a lifelong learner, I remain committed to exploring emerging technologies, enhancing my skill set, and contributing to impactful innovations in the fields of Artificial Intelligence and Software Engineering.

Muhammad Anas Rathore